1. Why does BFS ignore the distances stored in the graph even though they are available to it?

   BFS optimizes for hop count (number of edges), not path cost. By design, standard BFS uses a FIFO queue where edge traversal costs are implicitly treated as uniform ($c = 1$). Because it assumes all transitions carry equal cost, it only tracks the depth level rather than querying or accumulating physical edge weights.

2. Which two lines change when bfs is turned into dfs, and which of them makes DFS order dependent?

   - **Popping from the frontier:** changing frontier.popleft() (FIFO queue) to frontier.pop() (LIFO stack).
   - **Adding/pushing neighbours to the frontier:** changing queue append logic to stack push logic (or iterating through neighbours).
   - **Which line causes order dependence: The neighbour addition/ordering line** (or how neighbours are pushed/iterated onto the stack). Because a stack processes elements in Last-In-First-Out order, reversing or altering the iteration order over adjacent nodes completely changes which path DFS plunges into first (as shown in your terminal where original vs. reversed order yields entirely different hop counts: 23 vs. 8 hops).

3. UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?

   UCS would return a suboptimal path. Testing upon generation can prematurely accept a newly generated path before cheaper alternate routes reaching the same goal are explored. By delaying the goal test until popping from the priority queue, UCS guarantees that no unexplored path currently on the frontier has an accumulated path cost $g(n)$ less than the goal state.

4. Lab1reports *expanded* and *runtime_ms*. Which one should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?

   Use expanded. The number of nodes expanded is a deterministic, machine-independent metric that reflects purely algorithmic behaviour and graph structure. In contrast, runtime_ms is heavily influenced by differences in hardware specifications, background OS tasks, CPU clock speeds, and memory performance.

5. Which quantity does GBFS ignore that A⋆ uses, and which failure in Lab2 does that explain?

   GBFS ignores the path cost accumulated so far ($g(n)$), relying strictly on heuristic distance to the goal ($h(n)$). This explains the failure where GBFS finds a significantly suboptimal path (or gets misdirected into excessive exploration/dead ends by greedy local choices) compared to A^* and UCS, which balance $h(n)$ with true travel distance $g(n)$.

6. Frontier states *X* and *Y* have *(g, h) = (8,2)* and *(3,6)*. Which does UCS expand first? GBFS? A⋆?

   Evaluating each priority function for states $X(g = 8, h = 2)$ and $Y(g = 3, h = 6)$:

- UCS evaluates $f(n) = g(n)$:
  - $f(X) = 8$
  - $f(Y) = 3$
  - Expands Y first.
- GBFS evaluates $f(n) = h(n)$:
  - $f(X) = 2$
  - $f(Y) = 6$
  - Expands X first.
- A* evaluates $f(n) = g(n) + h(n)$:
  - $f(X) = 8 + 2 = 10$

- $f(Y) = 3 + 6 = 9$
- Expands Y first.

7. Why must h be expressed in the same units as the edge costs?

   Heuristic values and edge costs are directly added together in the evaluation function ($f(n) = g(n) + h(n)$). If h and g were measured in different units (e.g., flight kilometers vs. flight hours), one quantity would arbitrarily dominate or scale improperly, destroying the admissibility ($h(n) \leq h^*(n)$) and consistency properties needed to guarantee optimality.

8. Lab2 computes $h\star$ by running Dijkstra on the reversed graph from the goal. Why reversed?

   In a directed graph (such as flight routes), outgoing edges from the goal represent flights departing from it, not arriving at it. Reversing every directed edge ($u \to v$ becomes $v \to u$) and running a single-source Dijkstra from the goal node calculates the true shortest distance from all airport nodes to the goal simultaneously in $\mathcal{O}(E + V \log V)$ time, producing the exact ground-truth cost-to-go $h^*(n)$.

9. Weighted A$\star$ with $w = 1.5$ changed the cost for KUL→KEF but not for SIN→EZE. What property of the SIN→EZE route explains this?

   The optimal route for SIN → EZE already aligns closely with the direction of the heuristic, meaning the greedy path suggested by $h(n)$ happens to coincide with the globally optimal lowest-cost path. In contrast, for KUL → KEF, weighting h pulls the search toward locally attractive shortcuts that trade off true overall flight distance, leading Weighted A* to sacrifice cost optimality for speed.